

5.

La riflessione

Marco Faella

Dip. Ing. Elettrica e Tecnologie dell'Informazione
Università di Napoli “Federico II”

Corso di Linguaggi di Programmazione II

- La riflessione (o *introspezione*) è quella caratteristica di un linguaggio che permette ai programmi di **investigare a tempo di esecuzione sui tipi effettivi** degli oggetti manipolati
- Java fornisce ampio supporto alla riflessione
- Il cardine della riflessione è rappresentato dalla **classe Class**
- Ciascun oggetto di classe Class rappresenta una delle classi del programma
 - la JVM si occupa di istanziare un oggetto Class per ogni nuova classe caricata in memoria
 - solo la JVM può istanziare la classe Class
 - un oggetto di tipo Class contiene tutte le informazioni relative alla classe che esso rappresenta: i costruttori, i metodi, i campi, ed anche le eventuali classi interne
 - tramite questo oggetto, è possibile conoscere a run-time le caratteristiche di una classe che non è nota al momento della compilazione
- Pur non essendo classi, anche i tipi primitivi hanno un corrispondente oggetto di tipo Class

Per ottenere un riferimento ad un oggetto di tipo Class, è possibile utilizzare tre tecniche:

- 1) Il metodo **getClass** della classe Object
- 2) L'operatore **.class**
- 3) Il metodo statico **forName** della classe Class

- Nella classe Object, è presente il metodo

```
public Class<?> getClass()
```

- Questo metodo restituisce l'oggetto Class corrispondente al tipo **effettivo** di questo oggetto

- Ad esempio:

```
Employee e = new Manager(...);
```

```
Class<?> c = e.getClass();
```

A questo punto, c conterrà un riferimento all'oggetto Class della classe Manager

- Il **tipo restituito** di getClass merita attenzione (tra qualche slide...)

- La classe Class ha un parametro di tipo, che chiameremo "T"
- Come un serpente che si morde la coda, il parametro di tipo di un oggetto Class indica il tipo che questo oggetto rappresenta
- Ad esempio, se x è l'oggetto Class relativo alla classe Employee, il tipo di x è `Class<Employee>`
- Nelle prossime slide, vedremo come il parametro di tipo viene utilizzato dai metodi di Class

- Esaminiamo alcuni metodi della classe `Class<T>`:

`public String getName()`

- Restituisce il nome di questa classe, completo di eventuali nomi di pacchetti (*fully qualified*)

`public T newInstance()`

- Crea e restituisce un nuovo oggetto di questa classe, invocando un costruttore senza argomenti, che questa classe deve possedere

`public static Class<?> forName(String name)`

- Restituisce l'oggetto `Class` corrispondente alla classe di nome "name"; la stringa "name" deve contenere anche l'indicazione degli eventuali pacchetti cui la classe appartiene

`public Class<? super T> getSuperclass()`

- Restituisce l'oggetto `Class` corrispondente alla superclasse diretta di questa; se questa è `Object`, oppure è un'interfaccia o un tipo base, il metodo restituisce `null`

`public boolean isInstance(Object x)`

- Restituisce vero se (e solo se) il tipo effettivo di `x` è sottotipo di questa classe
- E' la versione riflessiva di `instanceof` (dinamica anche rispetto all'operando destro)

- Supponiamo di applicare getClass ad un oggetto di tipo dichiarato Employee:

```
Employee e = ... ;  
??? = e.getClass();
```

- Qual è il tipo *più specifico* a cui possiamo assegnare il risultato della chiamata?

- Supponiamo di applicare getClass ad un oggetto di tipo dichiarato Employee:

```
Employee e = ... ;  
??? = e.getClass();
```

- Qual è il tipo *più specifico* a cui possiamo assegnare il risultato della chiamata?
- A prima vista, la risposta sembrerebbe Class<Employee>
- A pensarci meglio, se il tipo effettivo di "e" fosse Manager, l'oggetto restituito da getClass sarebbe di tipo Class<Manager>, che non è assegnabile a Class<Employee>!
- Pertanto, il tipo più appropriato (cioè, più specifico) a cui è ragionevole assegnare il risultato è:

Class<? extends Employee>

- Ma Class<?> (tipo di ritorno di getClass) **non** è assegnabile a Class<? extends Employee>

Come fare?

- Il tipo restituito di getClass dovrebbe esprimere il seguente concetto:

*Applicato ad un'espressione di tipo dichiarato A,
questo metodo restituisce un oggetto di tipo Class<? extends A>*

- Purtroppo, **non è possibile** esprimere in Java questa proprietà
- Quindi, il type-checker tratta il metodo getClass **in modo particolare**, simulando quel tipo restituito, che il linguaggio non è in grado di esprimere

- Nota: Fino a Java 5 (aka 1.5), il tipo di ritorno di getClass era dichiarato "Class<? extends Object>"

- TypeScript è un'estensione fortemente e staticamente tipata di Javascript
- In TypeScript è possibile indicare il tipo dell'oggetto corrente con una keyword (this)

```
class Box {  
  contents: string = "";  
  set(value: string): this {  
    this.contents = value;  
    return this;  
  }  
}
```

Indica il *tipo* di this

```
class ClearableBox extends Box {  
  clear() {  
    this.contents = "";  
  }  
}
```

```
const a: ClearableBox = new ClearableBox();  
const b: ClearableBox = a.set("hello");
```

Quindi, se Java potesse indicare il tipo dell'oggetto corrente con una keyword (ad es., *thisType*), nella classe `Object`, avremmo:

```
public boolean equals(thisType other) { ... }  
public Class<thisType> getClass() { ... }
```

Invece, abbiamo:

```
public boolean equals(Object other) { ... }  
public Class<?> getClass() { ... }
```

- Precisamente, il tipo di ritorno di `<exp>.getClass()` è l'*erasure* del tipo dichiarato dell'espressione `<exp>`
- Quindi, l'esempio seguente **non compila**:

```
public static <T> Class<? extends T> myGetClass(T x) {  
    return x.getClass(); // errore qui  
}
```

Infatti:

- Il tipo dichiarato di `x` è `T`
- L'*erasure* di `T` è `Object`
- Il tipo di ritorno di `x.getClass()` è `Class<? extends Object>`
- `Class<? extends Object>` non è assegnabile a `Class<? extends T>`

- La seconda tecnica per ottenere un riferimento ad un oggetto di tipo Class sfrutta l'operatore **".class"**
- Tale operatore si applica al **nome di una classe** o di un tipo primitivo, come in:

Employee.class

String.class

java.util.LinkedList.class

int.class

- Pertanto, è un *operatore unario postfixo*

- L'operatore .class ha carattere *statico*, nel senso che il suo valore è **noto al momento della compilazione**
- Quindi, applicato ad un nome di classe A, forma un'espressione di tipo `Class<A>`
- Ad esempio:

```
Class<String> c1 = String.class;  
Class<Integer> c2 = Integer.class;  
Class<Integer> c3 = int.class;
```

Nota: nell'esempio qui sopra c2 e c3 sono due oggetti differenti

- La terza tecnica è rappresentata dal metodo statico forName della classe Class, già presentato in questa lezione
- Questa tecnica ha carattere *dinamico*, in quanto il valore restituito da forName non è noto al momento della compilazione

Data la dichiarazione:

```
Employee e = new Manager(...);
```

Per ognuna delle seguenti espressioni, dire se compila o meno, e in caso affermativo calcolarne il valore

- 1) `e instanceof Employee`
- 2) `e instanceof Manager`
- 3) `e.class instanceof Employee`
- 4) `e.getClass() == Manager`
- 5) `e.getClass() == Employee.class`
- 6) `e.getClass() == "Manager" (*)`
- 7) `e.getClass() == Manager.class`
- 8) `e.getClass().equals(Manager.class)`

(*) La risposta dipende dalle regole di tipo dell'operatore "=="

Realizziamo un metodo statico "fill", che accetta un array e un oggetto Class e riempie l'array di nuove istanze della classe corrispondente

Ecco una possibile invocazione:

```
Employee[] a = new Employee[10];  
Test.<Employee>fill(a, Manager.class);
```

Questo frammento dovrebbe riempire l'array con 10 nuovi oggetti Manager, creati con il costruttore senza argomenti di Manager (se esiste)

Implementazione, all'interno di una classe Test:

```
public static <T> void fill(T[] arr, Class<? extends T> c)
    throws InstantiationException, IllegalAccessException
{
    for (int i=0; i<arr.length; i++) {
        T x = c.newInstance();
        arr[i] = x;
    }
}
```

Il metodo `newInstance` può lanciare diverse **eccezioni verificate**, ad esempio nel caso in cui la classe in questione non possenga un costruttore senza argomenti, o se tale costruttore non sia accessibile.

Si noti il tipo del parametro `Class`, che consente di istanziare oggetti di una sottoclasse del tipo dell'array.

Attenzione: purtroppo, anche la seguente invocazione è lecita a tempo di compilazione:

```
Employee[] a = new Employee[10];
Test.<Object>fill(a, String.class)
```

Esplorare il contenuto di una classe

- I metodi della classe `Class` permettono di ricavare numerose informazioni sulla classe in questione
- In particolare, è possibile conoscere l'elenco di tutti i **campi**, **metodi** e **costruttori** appartenenti alla classe
- A tale scopo, esistono le classi **Field**, **Method** e **Constructor**, che rappresentano gli elementi omonimi di una classe

- Per ottenere tali informazioni, sono utili i seguenti metodi di Class:

<code>public Field[] getFields()</code>	Tutti i campi pubblici (anche ereditati)
<code>public Field[] getDeclaredFields()</code>	Tutti i campi dichiarati qui (anche privati)

<code>public Method[] getMethods()</code>
<code>public Method[] getDeclaredMethods()</code>

<code>public Constructor[] getConstructors()</code>
<code>public Constructor[] getDeclaredConstructors()</code>

- Il metodo `getFields` restituisce tutti i campi **pubblici** di questa classe, anche ereditati
 - `getMethods` è analogo a `getFields`
 - `getConstructors` restituisce semplicemente i costruttori pubblici di questa classe
- Il metodo `getDeclaredFields` restituisce tutti i campi dichiarati in questa classe (e non nelle superclassi), indipendentemente dalla loro visibilità
 - similmente, `getDeclaredMethods` e `getDeclaredConstructors`

- La classe **Field** rappresenta un **campo** di una classe
- Essa dispone di metodi per leggere e modificare il contenuto di un campo, conoscere il suo nome e il suo tipo
- In particolare, abbiamo:

public String getName()	Restituisce il nome di questo campo.
public Object get (Object x) throws IllegalAccessException	Restituisce il valore di questo campo nell'oggetto x. Se questo campo è di un tipo base, il suo valore viene racchiuso nel corrispondente tipo riferimento (ad es., int -> Integer). Se questo campo è statico, il parametro x viene ignorato.
public void set (Object x, Object val) throws IllegalAccessException	Imposta a val il valore di questo campo nell'oggetto x. Se questo campo è statico, il parametro x viene ignorato.
public Class<?> getType()	Restituisce il tipo di questo campo.

- Alcuni metodi sollevano l'eccezione verificata `IllegalAccessException`, quando si tenta di accedere ad un campo che non è accessibile a causa della sua visibilità

- I metodi get e set di Field (così come altri metodi simili delle classi Method e Constructor) applicano le regole di visibilità previste dal linguaggio
- Cioè, lanciano l'eccezione verificata `IllegalAccessException` se si tenta di accedere ad un campo che non è visibile dalla classe in cui ci si trova
- E' possibile disattivare questo controllo utilizzando i metodi della classe **`AccessibleObject`**, superclasse comune a Field, Method e Constructor
- La possibilità di aggirare i controlli è soggetta al Security Manager, l'oggetto che è responsabile dei permessi per le operazioni a rischio
- Di default, la JVM si avvia senza un Security Manager, consentendo alle applicazioni di compiere qualsiasi operazione
- Invece, i browser che eseguono applet Java utilizzano dei Security Manager particolarmente restrittivi

- La classe **Method** rappresenta un metodo di una classe
- Essa dispone di metodi per conoscere il nome del metodo, il numero e tipo dei parametri formali e il tipo di ritorno
- Inoltre, è possibile invocare il metodo stesso
- In particolare, abbiamo:

public String getName ()	Restituisce il nome del metodo rappresentato.
public Object invoke (Object x, Object...args) throws IllegalAccessException	Invoca sull'oggetto x il metodo rappresentato, passandogli i parametri attuali args. Se il metodo rappresentato è statico, il parametro x viene ignorato. Restituisce il valore restituito dal metodo rappresentato.

- La sintassi "Object...args" indica che invoke accetta un numero variabile di argomenti

Metodi variadici

Metodi con numero variabile di argomenti

- Dalla versione 1.5, Java prevede un meccanismo per dichiarare metodi con un numero variabile di argomenti (metodi *variadici*, o, in breve, *varargs*)
- Se T è un tipo di dati, con la sintassi

`f(T ... x)`

- si indica che f accetta un numero variabile di argomenti (anche zero), tutti di tipo T
- I puntini sospensivi devono essere necessariamente *tre*
- Gli argomenti possono essere passati separatamente, come in `f(x1, x2, x3)`, oppure tramite un array, come in `f(new T[] {x1, x2, x3})`
- All'interno del metodo f, si accede agli argomenti utilizzando x come un array di tipo T
- Ogni metodo può avere un solo argomento variadico, che deve essere l'**ultimo** della lista, come il metodo *invoke* della classe Method, illustrato nella slide precedente

```
public Object invoke(Object x, Object...args)
```

In altri linguaggi

- Il **C** non fornisce alcun supporto alla riflessione
 - Nota: `sizeof` è un operatore valutato a *tempo di compilazione*
- Il **C++** fornisce un supporto parziale alla riflessione, chiamato **Run-Time Type Information (RTTI)**
- Diremo che un oggetto “conosce il proprio tipo” se a partire dal suo indirizzo è possibile risalire all’identità del suo tipo
- Ovvero, se nel *memory layout* dell’oggetto è presente un puntatore o un identificativo della sua classe
- In C, nessun “oggetto” (ad es., `struct`) conosce il proprio tipo
- In Java, tutti gli oggetti conoscono il proprio tipo
- In C++, RTTI agisce solo su classi che hanno *almeno un metodo virtuale* (cioè sovrascrivibile)
 - Gli oggetti di queste classi conoscono il proprio tipo, per permettere il binding dinamico
 - Gli oggetti delle altre classi non conoscono il proprio tipo

- **Operatore typeid:** typeid(exp)
 - Restituisce un oggetto di tipo std::type_info corrispondente al tipo *dichiarato o effettivo* di "exp"
 - Effettivo, se il tipo dichiarato di exp conosce il proprio tipo (simile a getClass)
 - Dichiarato, altrimenti
- **Dynamic cast:** dynamic_cast<type>(exp)
 - Un cast che controlla a runtime se "exp" è di tipo effettivo "type"
 - Se non lo è, restituisce nullptr (in caso di tipo puntatore) o lancia un'eccezione (in caso di tipo riferimento)
 - Non compila se il tipo dichiarato di exp non conosce il proprio tipo
 - Simile a un cast Java tra riferimenti

```
class A { };  
class B : public A { };
```

Le classi A e B *non* conoscono il proprio tipo

```
class C {  
    virtual void foo() {}  
};  
class D : public C { };
```

Le classi C e D conoscono il proprio tipo

```
int main(void) {  
    A a;  
    B b;  
    A *p = &b;  
  
    cout << typeid(a).name() << endl;  
    cout << typeid(b).name() << endl;  
    cout << typeid(*p).name() << endl;  
  
    D d;  
    C *q = &d;  
    cout << typeid(*q).name() << endl;  
}
```

Output:

1A

1B

1A

1D

- 1) Implementare un metodo, chiamato *reset*, che prende come argomento un oggetto ed imposta a zero tutti i suoi campi interi pubblici
- 2) Implementare un metodo che, dato un oggetto, parte dalla classe che rappresenta il tipo effettivo dell'oggetto e ne restituisce la superclasse più generale, escludendo Object (quindi, la penultima classe, prima di arrivare a Object)